Ruby doesn't have the autoincrement (++) and autodecrement (–) operators that C, Java, and JavaScript have. Use the += and -= forms instead.

Conditional Execution

Ruby has several different mechanisms for conditional execution of code; they should feel similar to other programming languages, but many have some neat twists. Before we get into them, though, we need to spend a short time looking at boolean expressions.

Boolean Expressions

Ruby has a simple definition of truth. Any value that is not nil or the constant false is true—"cat", 99, 0, and :a_song are all considered true. An empty string, "", an empty array [], and an empty hash {} are all true in Ruby. (You will sometimes see Rubyists refer to the set of all false values as "falsey" and the set of all true values as "truthy")

In this book, when we want to talk about a general true or false value, we use regular Roman type: true and false. When we want to refer to the actual constants, we write true and false.

The fact that nil is considered to be false is convenient. For example, gets, which returns the next line from a file, returns nil at the end of file, enabling you to write loops such as this:

```
while (line = gets)
 # process line
end
```

And, Or, and Not

Ruby supports all the standard boolean operators. Both the keyword and and the operator && (logical and) return their first argument if it is falsey. Otherwise, they evaluate and return their second argument (this is sometimes known as *short circuit* evaluation).

```
nil && 99   # => nil
false && 99 # => false
"cat" && 99 # => 99
```

The only difference in the two forms is precedence—the && has higher precedence than and, meaning that where there's a choice of operators to evaluate, && will be evaluated first but and will be evaluated last.

```
result = "a" && "b"
result    # => "b"

result = "a" and "b"
result    # => "a"
```

In the first line, the operator && has higher precedence than the assignment, so it is evaluated first, returning "b", and result is set to "b", as if the line was written result = ("a" && "b").

In the second line, the assignment has higher precedence than and, so result is set to "a" first, then the and is evaluated, as if it was (result = "a") and "b" – the entire sequence still returns "b", but in the second line, the result is set to "a". We strongly recommend that you use parentheses in any case where the order of execution might be ambiguous or confusing.

The && and and operators return a true value only if both of their arguments are true.